CMPUT 291 Mini Project 1 Design Doc (OOPS-we-coded-again)

General Overview / User Guide

After calling the python file with a valid database file path, the user is prompted to sign in, register or exit.

- Sign in: input a valid user ID and password
- Register: Create a new account with a valid name, email, and phone number. The user is then given a unique user ID.
- Exit: close the program

After signing in or registering, the user is placed on their home screen and shown any new tweets in their feed (5 at a time). Here, the user can view more tweets in their feed, search tweets or users, compose tweets, list their followers, or logout

- View more tweets: view the next batch of tweets in the user's feed (up to 5 at a time)
- Search tweets: search for tweets with text or hashtag matches
    - The user can view info about these tweets, view the next page of results, retweet them, reply to them, or go back to the home screen
- Search users: search for a user by name
    - The user can view the next page of results, view the info of a user, or go back to the home screen
        - If viewing another user's info, the current user can view their tweets, follow them, or return to the search screen
- Compose tweet: the user can type a message to tweet, and the message can include a hashtag. The user can also return to the home screen
- List followers: the user can view a list of their followers
    - They can view info of a follower, see the next page of followers, or return to the home screen
- Logout: the user logs out of the program, returning to the login / register / exit page

System Architecture

[User Interface Layer]
↓ ↑
[Core Functions Layer]
- User Management
- Tweet Operations
- Search Functions

↓ ↑
[Database Layer]
SQLite Database

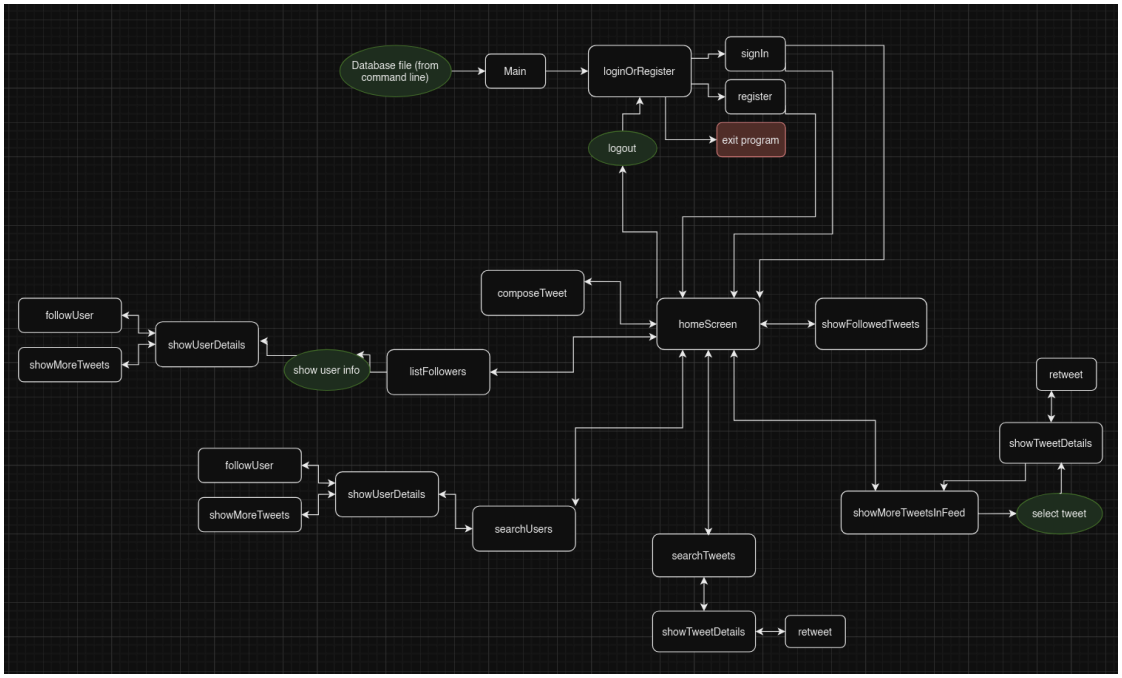

Diagram of the functions and their paths

Design of Software

main(): handles database path input and validation and initiates a connection to the database. Then initiates the program (**loginOrRegister()**)

connect(): connects the user to the database provided.

loginOrRegister(): handles user input and allows the user to initiate logging in (**signIn()**), registering (**register()**), or exiting the program. After the user logs in or registers, this function takes them to the home screen (**homeScreen()**).

signIn(): lets the user sign in by tying a valid user ID and password

register(): handles user input of a valid name, email, and phone number, then creates a unique user ID and adds them to the database of users and signs them in

homeScreen(): handles users input, calls **showFollowedTweets()** to show them their feed, and allows the user to start a new action (possible actions: show more tweets (**showMoreTweetsInFeed()**), search tweets (**searchTweets()**) / users (**searchUsers()**), compose a tweet (**composeTweet()**), list followers (**listFollowers()**), or logout (**signinOrRegister()**)).

showMoreTweetsInFeed(): displays the next batch of up to five tweets in their feed based on their current position in the feed

showFollowedTweets(): display the first 5 tweets in your feed

searchTweets(): search for a tweet with keywords/hashtags, then selects matching tweets and displays them. Then allows the user to view tweet details (**showTweetDetails()**).

showTweetDetails(): shows info about a selected tweet and provides the user options to reply or retweet (**retweet()**) the tweet

retweet(): retweets a tweet. Marks the retweet as spam if already retweeted.

searchUsers(): allows the user to search for a user by name. Then shows a list of users and allows them to show details of a user (**showUserDetails()**).

showUserDetails(): shows the followers (**listFollowers()**) and recent tweets of a user and allows the current user to follow them or see the next page of recent tweets (**showMoreTweets()**).

listFollowers(): lists the followers of a user.

followUser(): follows the selected user.

showMoreTweets(): show additional tweets in the user info screen

composeTweet(): creates a tweet and adds it to the tweets table, and if the tweet has a hashtag, adds that to the hashtag table

checkRepeatedHashtags(): checks if there are any duplicate hashtags in a tweet (case-insensitive)

Testing Strategy

We each made sure to test our code as we went, spending a few minutes testing each new feature added. Before pushing an update to GitHub, we more thoroughly tested our new code to make sure it worked. However, this did not always catch everything, so we made sure to keep in constant contact with each other and reported any bugs found in someone else's code. If a bug was found in someone's code, they would inform the team they were fixing it and would update the GitHub with fixed code. After the program was finished development, we each spent time thoroughly testing each part of the code. This included bad inputs, repeated behavior, testing navigation flow and adding/testing edge-case entries in the database file.

Group Work Breakdown Strategy

Work done by each member (along with a time estimate):

Sydney (time estimate: 9 hours):

- Created and set up the python file with SQL communication
- Handled command-line arguments and command-line argument / database validation
- Created the initial sign-in / register / exit screen
- Programmed the sign-in and register functions, along with input validation and unique user ID assignment
- Created the home screen command loop
- Created the design document

Utsha (time estimate: 7 hours):

- Created the ability for users to post new tweets
- Created the ability to reply to a tweet
- Created the system to detect and store hashtags in tweets
- Tested the code thoroughly and caught multiple bugs

Raiyana (time estimate: 15 hours):

- Created the newsfeed
- Created pagination code for Newsfeed
- Created the system to search for tweets with multiple keywords and hashtags
- Added pagination for search results
- Implemented detailed view of tweets
- Added reply and retweet options
- Displayed tweet metadata (date, time, replies, retweets)
- Created retweet functionality
- Created the system to set spam flags for duplicate retweets
- Managed database operations for retweets
- Implemented tweet ID generation logic in the composeTweet()
- Implemented navigation between different screens
- Handled error cases and edge conditions

Faiaz (time estimate: 8 hours):

- Created the system to search for users and view extra stats of the users
- Created the list followers functionality where the user can see the stats of the followers and follow them back
- Created the pagination code used throughout the project
- Created the system to follow users

To coordinate our work, we used a combination of GitHub, Instagram, and Google Docs. We used GitHub as a place to keep our code, which we kept in separate branches while being worked on. The code was then merged into the main branch after our part was completed. We used Instagram to communicate throughout the project, notifying other members of bugs found in the code, checking for progress updates, and discussing testing methods. Our Google Doc was used to outline the work each of us needed to do.